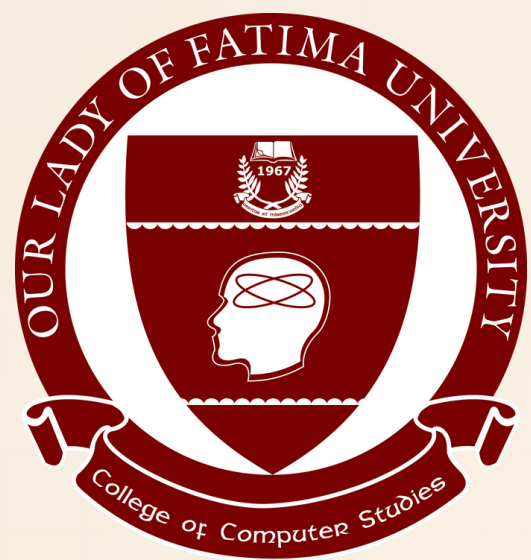




Module 5

Operating System

ITCL112 - INTRODUCTION TO COMPUTING W/ LAB

LEARNING OUTCOME



- **Explain the purpose of an operating system and its main functions (resource, process, file, and user management).**
- **Compare major operating systems, their features, advantages, and limitations.**
- **Demonstrate file organization and operations, and distinguish between different types of user interfaces (CLI, GUI, VUI, NUI).**
- **Identify installation requirements and explain the importance of updates, patches, and upgrades for system performance and security.**

MODULE 5

- 1. ROLE AND FUNCTIONS OF AN OS**
- 2. COMMON OPERATING SYSTEMS USED TODAY
(WINDOWS, MACOS, LINUX)**
- 3. FILE MANAGEMENT AND USER INTERFACE**
- 4. UNDERSTANDING OPERATING SYSTEM
REQUIREMENTS AND UPDATES**



What is Operating System?



An operating system acts as an intermediary between the user of a computer and computer hardware. In short its an interface between computer hardware and user.

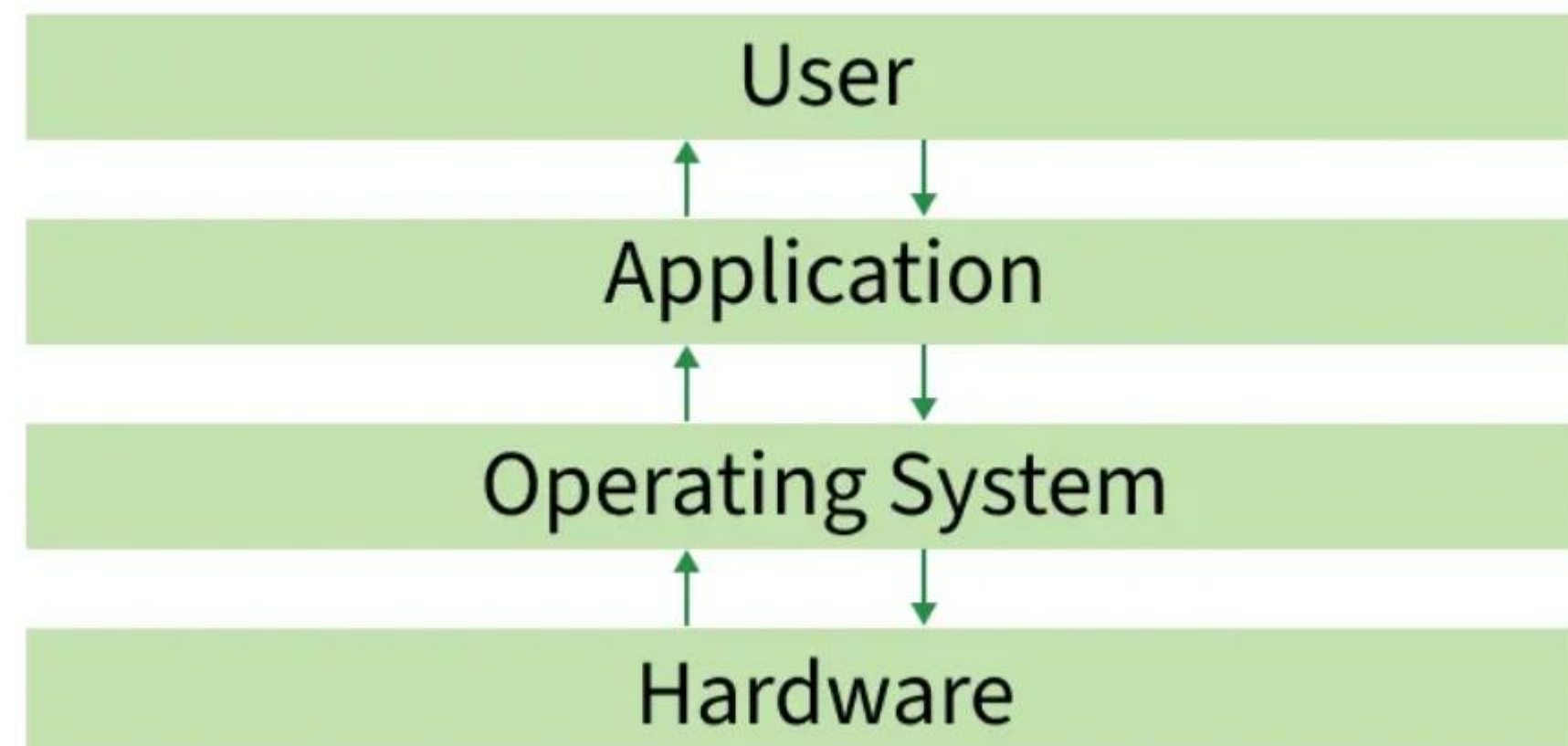
- **The purpose of an operating system is to provide an environment in which a user can execute programs conveniently and efficiently.**
- **An operating system is software that manages computer hardware and software. The hardware must provide appropriate mechanisms to ensure the correct operation of the computer system and to prevent user programs from interfering with the proper operation of the system.**
- **The operating system (OS) is a program that runs at all times on a computer. All other programs, including application programs, run on top of the operating system.**
- **It does assignment of resources like memory, processors and input / output devices to different processes that need the resources. The assignment of resources has to be fair and secure.**

Operating System as a User Interface:



1. User (Interacts with the system)
2. System and application programs (Software tools for the user)
3. Operating system (Manages hardware and software resources)
4. Hardware (Physical computing devices)

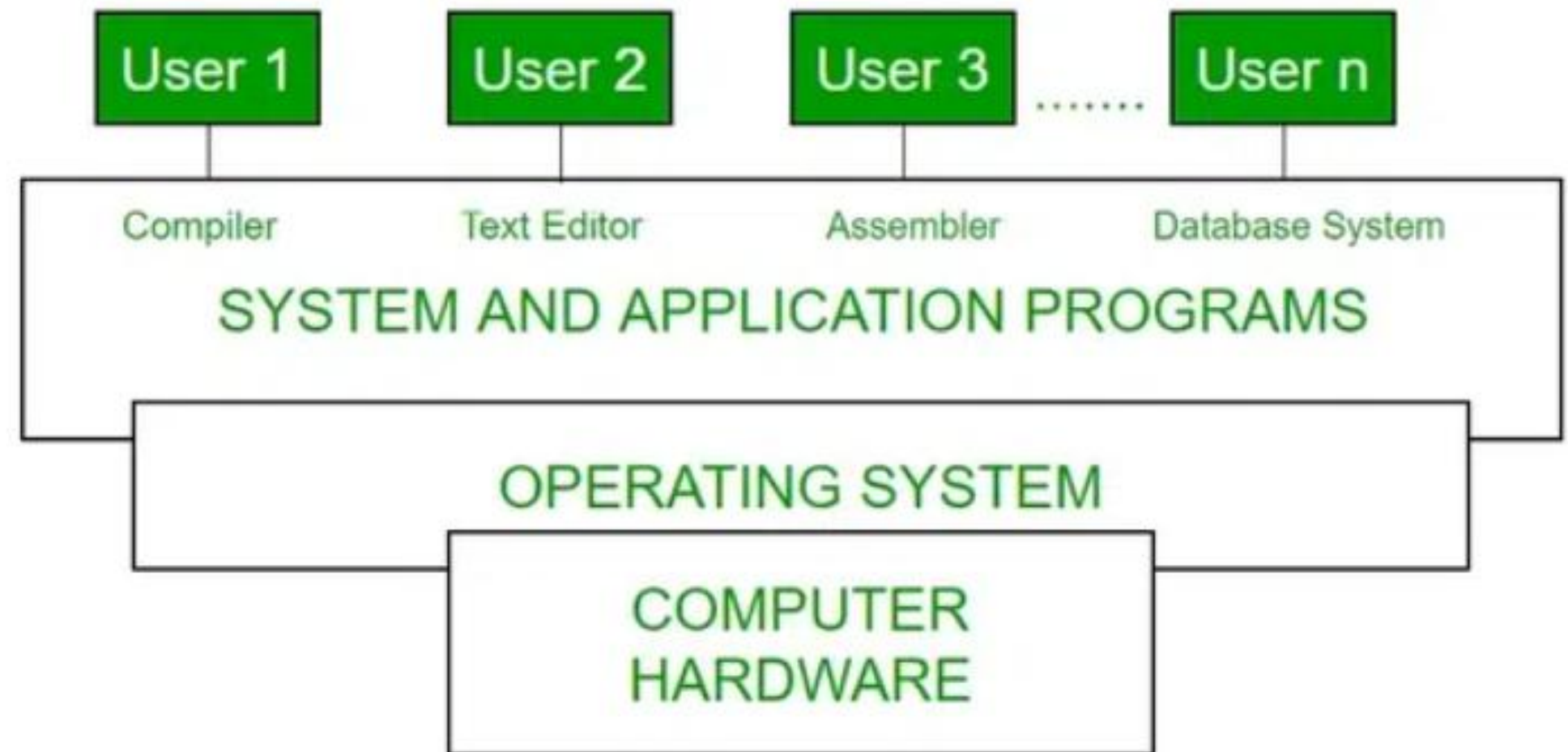
Control flow



Operating System as a User Interface:



Every general-purpose computer consists of hardware, an operating system(s), system programs and application programs. The hardware consists of memory, CPU, ALU, I/O devices, peripheral devices and storage devices. The system program consists of compilers, loaders, editors, OS, etc.



Goals of Operating System



Primary Goals

The primary goals of an operating system (OS) are to provide an easy to use and convenient environment for executing user programs.

1. **User Convenience** : It should be easy to use, providing a user-friendly interface and making it simple to interact with the system.
2. **Program Execution**: It facilitates the execution of user programs, providing the necessary environment and services for them to run.
3. **Resource Management**: The OS manages and allocates the computer's resources, including the CPU, memory, disk storage, and input/output devices, to ensure fair utilization.
4. **Security**: The OS protects the system and user data from unauthorized access, ensuring the confidentiality, integrity, and availability of information.

Goals of Operating System



Secondary Goals

1. **Efficient Resource Utilization:** It should aim to maximize the performance and utilization of computer resources like CPU, Memory and IO devices, ensuring that the system runs smoothly and efficiently.
2. **Reliability:** It should be robust and reliable, able to handle errors and exceptions gracefully, ensuring that the system continues to operate smoothly. It should be modular in design and easy to debug.

Role and Functions of an Operating System



Resource Management - Manages hardware resources such as CPU, memory, disk drives, and input/output devices.

Position Your Monitor – Controls running applications, allocates CPU time, and ensures multitasking without conflicts.

File Management – Provides a structured way to store, organize, and retrieve data.

User Interface (UI) – Allows users to interact with the computer through graphical interfaces (GUIs) or command-line interfaces (CLIs).

Security and Access Control – Protects data and system resources through authentication, permissions, and encryption.

Networking – Enables computers to communicate over local networks and the internet.

List of Common Operating Systems



Windows OS

- **Developer** : Microsoft
- **Key Features** : User-friendly interface, software compatibility, hardware support, Strong gaming support.
- **Advantages** : Easy to use for most users, Broad support from third-party applications ,Frequent updates and support.
- **Typical Use Cases** : Personal computing, Business environment, Gaming.

macOS

- **Developer** : Apple.
- **Key Features** : Sleek, intuitive user interface, Strong integration with other Apple products, Robust security features, High performance and stability.
- **Advantages** : Optimized for Apple hardware, Seamless experience across Apple ecosystem, Superior graphics and multimedia capabilities.
- **Typical Use Cases** : Creative industries (design, video editing, music production), Personal computing, Professional environments.

List of Common Operating Systems

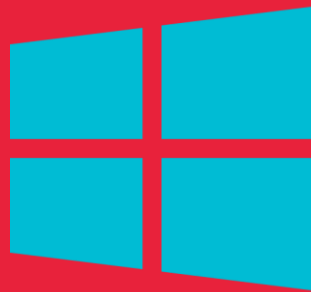


Linux

- **Developer** : Community-driven (various distributions).
- **Key Features** : Open-source and highly customizable, Robust security and stability, Lightweight and can run on older hardware, Large selection of distributions (e.g., Ubuntu, Fedora, Debian).
- **Advantages** : Free to use and distribute, Strong community support, Suitable for servers and development environments.
- **Typical Use Cases** : Servers and data centers, Development and programming, Personal computing for tech enthusiasts.

Unix

- **Developer**: Originally AT&T Bell Labs, various commercial and open-source versions available
- **Key Features**: Multiuser and multitasking capabilities, Strong security and stability, Powerful command-line interface, Portability across different hardware platforms
- **Advantages**: Reliable and robust performance, Suitable for high-performance computing and servers, Extensive support for networking
- **Typical Use Cases**: Servers and workstations, Development environments, Research and academic settings



Feature	Windows OS	macOS	Linux	Unix
Developer / Owner	Microsoft	Apple Inc.	Open-source community (with various distros like Ubuntu, Fedora, Debian, etc.)	Originally AT&T Bell Labs, now maintained by different vendors (IBM AIX, HP-UX, Solaris, etc.)
Source Model	Closed-source (proprietary)	Closed-source with open-source components	Open-source (GNU General Public License)	Mostly closed-source, though some open variants exist
Kernel Type	Hybrid kernel	Hybrid kernel (based on XNU: Mach + BSD)	Monolithic kernel (with modular support)	Monolithic kernel (originally)
User Interface (UI)	GUI (Windows Desktop, Start Menu) + CLI (Command Prompt, PowerShell)	GUI (Aqua interface, Finder, Dock) + Terminal (Unix-based)	Varies by distro: GUI (GNOME, KDE, etc.) + CLI (Bash, Zsh, etc.)	Primarily CLI (though GUIs can be layered, e.g., CDE, Motif)
System Architecture	Primarily x86, x86-64, ARM (newer versions)	Runs on Apple hardware (ARM-based Apple Silicon, Intel Macs)	Supports many architectures: x86, x86-64, ARM, PowerPC, etc.	Varies by vendor: x86, SPARC, PowerPC, etc.
Security	Frequent target for malware due to popularity; needs regular updates and antivirus	Strong security model, Unix-based foundation, Gatekeeper & sandboxing	High security, depends on distro and user configuration; less targeted by malware	Very secure, used mainly in enterprise and servers
Software Availability	Wide range: commercial apps, games, enterprise software	Limited to Apple ecosystem + some cross-platform apps	Wide open-source software library, fewer mainstream commercial apps	Limited third-party apps, mostly enterprise and specialized software
Cost	Paid (license fee, though bundled with PCs)	Paid (bundled with Apple hardware)	Free (most distros), paid enterprise support optional	Expensive (enterprise licenses)
Primary Use Cases	General consumer use, business, gaming, office productivity	Creative industries (design, video editing, music), Apple ecosystem integration	Servers, development, cybersecurity, embedded systems, enthusiasts	Servers, mainframes, mission-critical enterprise systems
Stability & Performance	User-friendly but can slow with bloat; frequent updates	Highly optimized for Apple hardware, stable	Very stable, scalable, customizable	Extremely stable, designed for high-performance enterprise workloads
Release Cycle	Frequent major releases + monthly security patches	Annual major releases + regular patches	Continuous updates (rolling or long-term support releases)	Vendor-specific (long, stable release cycles)

Characteristics of Operating Systems



- **Device Management:** The operating system keeps track of all the devices. So, it is also called the Input/Output controller that decides which process gets the device, when and for how much time.
- **File Management:** It allocates and de-allocates the resources and also decides who gets the resource.
- **Job Accounting:** It keeps track of time and resources used by various jobs or users.
- **Error-detecting Aids:** These contain methods that include the production of dumps, traces, error messages and other debugging and error-detecting methods.
- **Memory Management:** It is responsible for managing the primary memory of a computer, including what part of it are in use by whom also check how much amount free or used and allocate process
- **Processor Management:** It allocates the processor to a process and then de-allocates the processor when it is no longer required or the job is done.
- **Security:** It prevents unauthorized access to programs and data using passwords or some kind of protection technique.

Components of an Operating Systems



- **Shell** is the outermost layer of the Operating System and it handles the interaction with the user. It interprets input for the OS and handles the output from the OS.

2. Shell 🗨️ (the “translator” or “interpreter”)

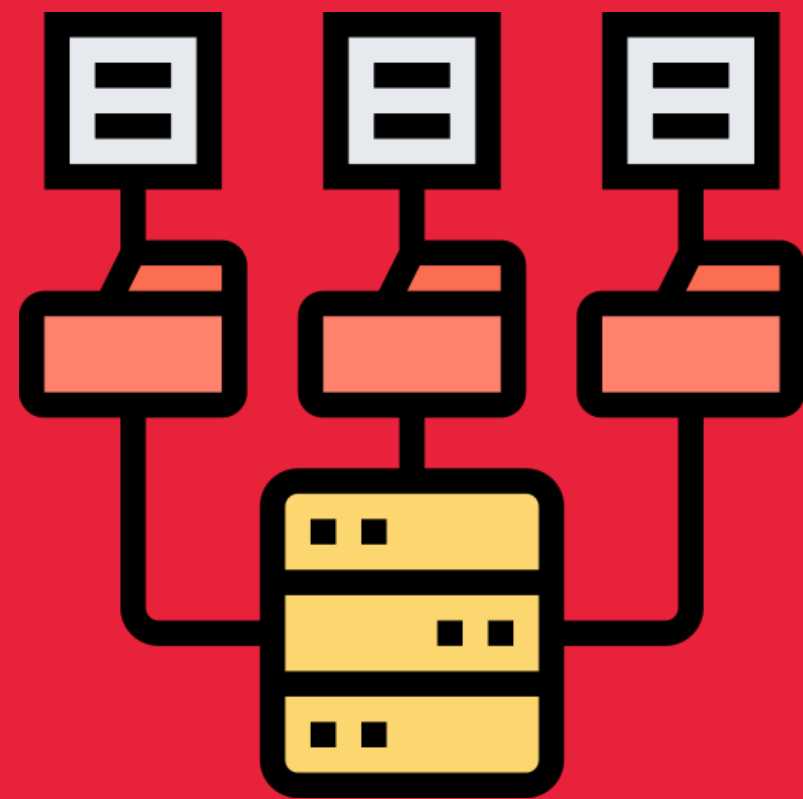
- The interface between the user and the kernel.
- Lets you communicate with the computer using commands or graphical menus.
- There are two main types:
 - Command-line shell (like typing commands in Windows Command Prompt or Linux Terminal).
 - Graphical shell (like clicking icons and menus in Windows or macOS).

- **Kernel** is the core component of operating system. The kernel is the primary interface between the Operating system and Hardware.

1. Kernel 🧠 (the “brain” of the OS)

- The core part of the operating system.
- It talks directly to the computer hardware (CPU, memory, disk, etc.).
- Manages resources: decides who gets to use the CPU, how memory is shared, how files are read/written.
- Example: When you press “Save” in a document, the kernel handles the actual saving to the hard drive.
- 👉 Think of the kernel as the engine of a car. You don’t see it working, but it makes the car run.

FILE MANAGEMENT



- Is the process of creating, naming, organizing, storing, retrieving, sharing, and protecting files on a computer or digital device.
- The OS organizes data in files and directories (folders).
- Uses a file system (e.g., NTFS(New Technology File System) for Windows, HFS (Hierarchical File System) for macOS, ext4 for Linux) to manage how files are stored on storage devices.

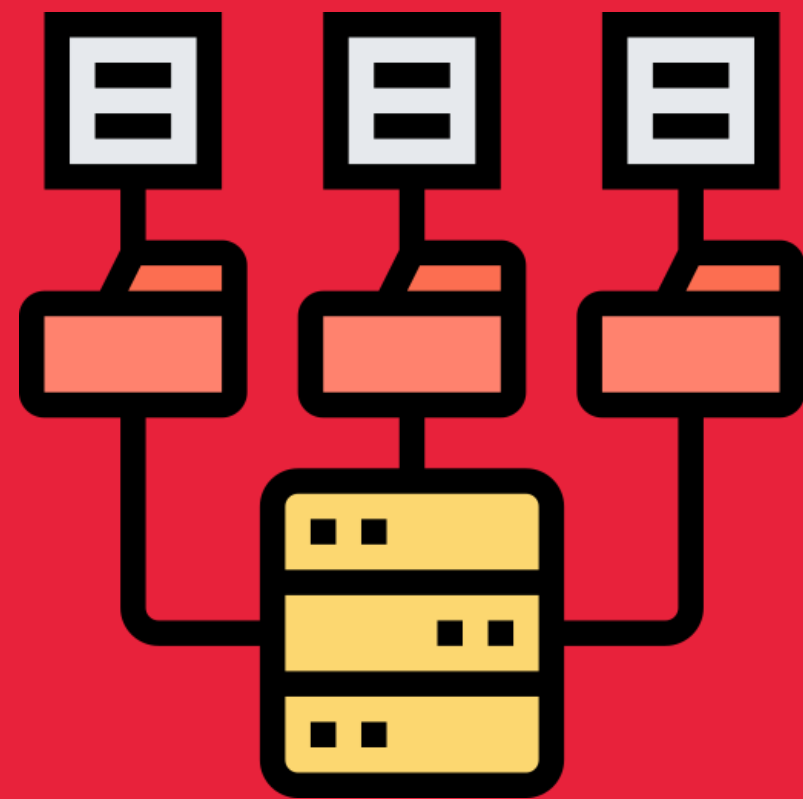
How a File System Works (Basic Idea)

A file system is like the “organizer” of your storage device. It decides where files go, how they are named, and how to find them later. Without it, your hard drive or USB would just be a big mess of 1s and 0s.

Think of it like a library system 📖:

- The books = your files (documents, pictures, videos).
- The shelves = folders/directories where files are grouped.
- The catalog system = the file system (it keeps track of where every book is stored so you can find it quickly).

FILE MANAGEMENT



- Is the process of creating, naming, organizing, storing, retrieving, sharing, and protecting files on a computer or digital device.
- The OS organizes data in files and directories (folders).
- Uses a file system (e.g., NTFS(New Technology File System) for Windows, HFS (Hierarchical File System) for macOS, ext4 for Linux) to manage how files are stored on storage devices.

How a File System Works (Basic Idea)

A file system is like the “organizer” of your storage device. It decides where files go, how they are named, and how to find them later. Without it, your hard drive or USB would just be a big mess of 1s and 0s.

Think of it like a library system 📚:

- The books = your files (documents, pictures, videos).
- The shelves = folders/directories where files are grouped.
- The catalog system = the file system (it keeps track of where every book is stored so you can find it quickly).

What is NTFS?



- NTFS stands for New Technology File System, and it's the default file system used by Windows operating systems (starting from Windows NT up to the latest versions). It's designed to be more secure, reliable, and efficient compared to older file systems like FAT16 and FAT32.
- **Large File and Volume Support** 📁
 - Can handle very large files (over 4 GB, unlike FAT32) and partitions up to several terabytes.
- **File Compression** 📦
 - Built-in ability to compress files and folders to save disk space.
- **Security Features** 🔒
 - Supports file- and folder-level permissions using Access Control Lists (ACLs).
 - Allows encryption with Encrypting File System (EFS) for data protection.
- **Fault Tolerance** ⚙️
 - Uses a transaction log (journaling) to recover data in case of a system crash.
- **Disk Quotas** 📊
 - Administrators can assign limits to how much disk space each user can use.
- **File System Reliability** ✅
 - Detects and repairs errors automatically.
 - Maintains metadata integrity.
- **File and Folder Management** 📁
 - Supports long file names, case sensitivity (optionally), and hard links.
- **Scalability and Efficiency** 🚀
 - Efficiently manages disk space with smaller cluster sizes compared to FAT systems.

Command-Line Interface

1. CLI (Command-Line Interface)

A text-based interface where the user types commands to perform tasks.

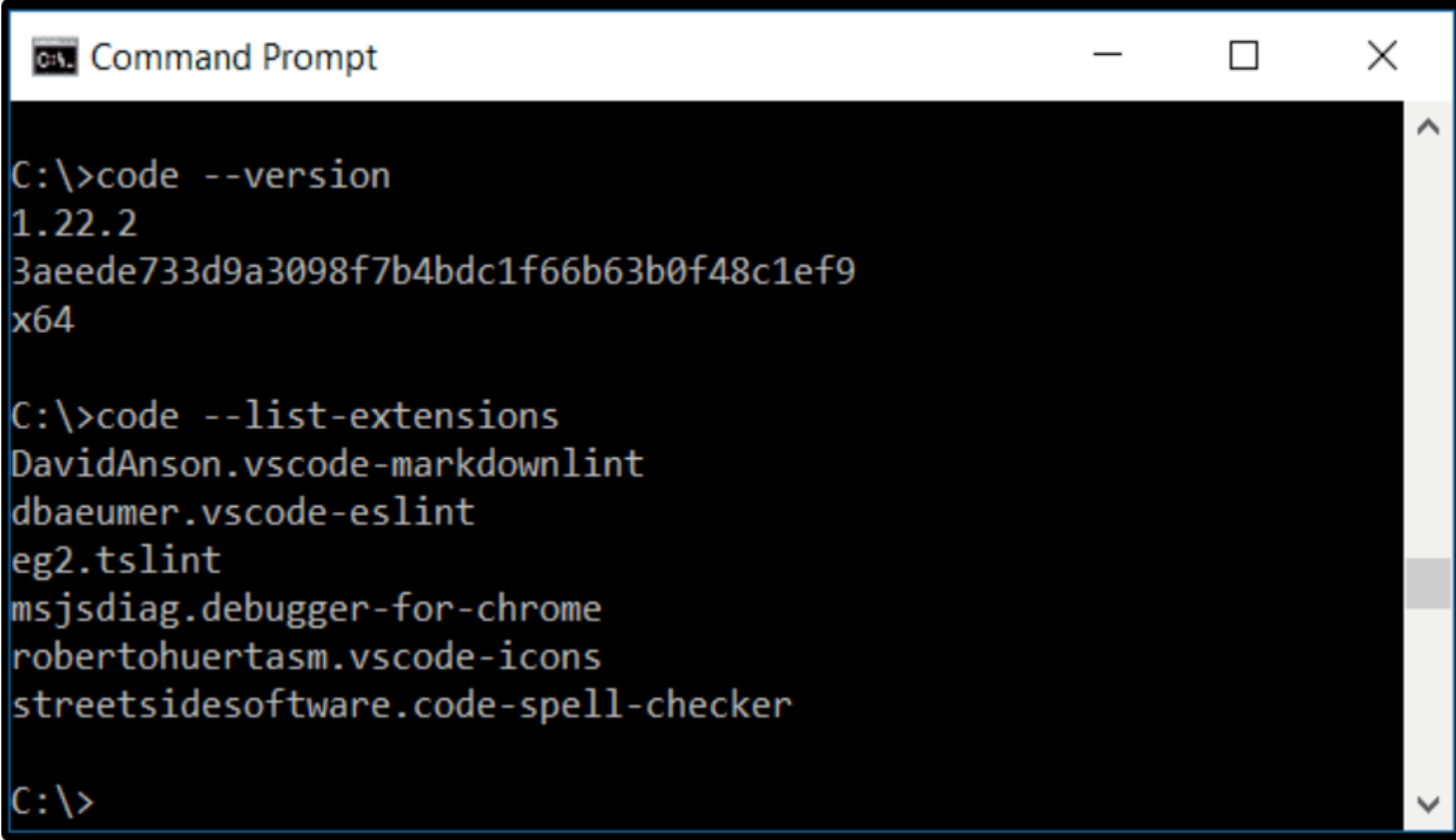
Example: Windows Command Prompt, Linux Terminal.

Advantages:

- Fast and powerful for experienced users.
- Requires fewer system resources.
- Allows automation using scripts.

Disadvantages:

- Steep learning curve (users need to remember commands). Not very user-friendly for beginners.

A screenshot of a Windows Command Prompt window titled "Command Prompt". The window has a black background and white text. It shows the output of two commands: "code --version" and "code --list-extensions". The first command outputs the version "1.22.2", a long hash "3aeede733d9a3098f7b4bdc1f66b63b0f48c1ef9", and the architecture "x64". The second command outputs a list of installed extensions: "DavidAnson.vscode-markdownlint", "dbaeumer.vscode-eslint", "eg2.tslint", "msjsdiag.debugger-for-chrome", "robertohuertasm.vscode-icons", and "streetsidesoftware.code-spell-checker". The prompt "C:\>" is visible at the bottom.

```
C:\>code --version
1.22.2
3aeede733d9a3098f7b4bdc1f66b63b0f48c1ef9
x64

C:\>code --list-extensions
DavidAnson.vscode-markdownlint
dbaeumer.vscode-eslint
eg2.tslint
msjsdiag.debugger-for-chrome
robertohuertasm.vscode-icons
streetsidesoftware.code-spell-checker

C:\>
```

Graphical User Interface (GUI)

2. GUI (Graphical User Interface)

A visual interface that uses windows, icons, menus, and pointers (WIMP) to interact with the system.

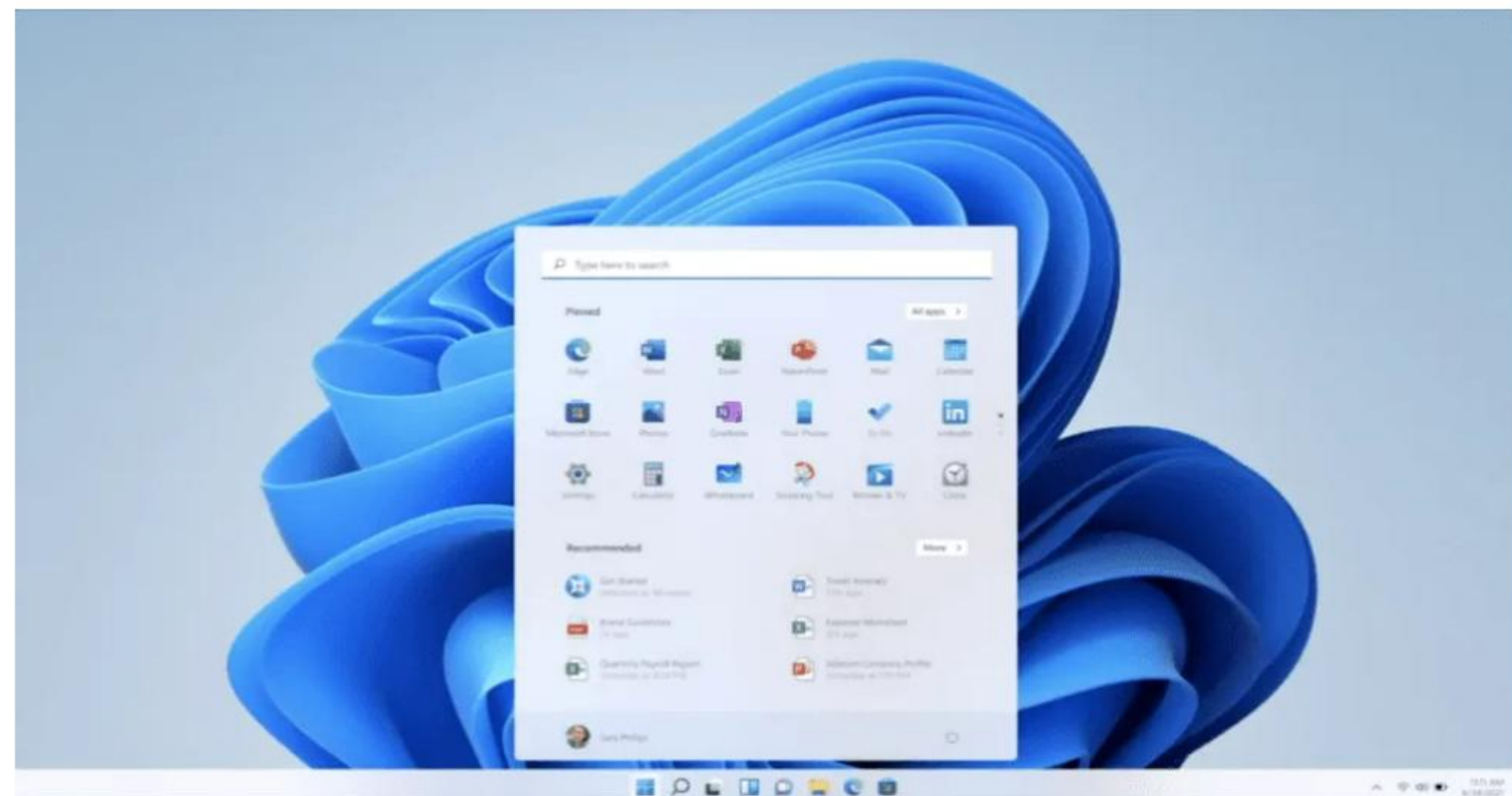
Example: Windows OS desktop, macOS, Android, iOS.

Advantages:

- Easy to use and learn.
- Intuitive uses visual elements.
- Supports multitasking (working with multiple windows at once).

Disadvantages:

- Consumes more memory and processing power.
- Can be slower than CLI for advanced tasks.



Menu-Driven Interface

3. Menu-Driven Interface

An interface where users navigate and choose options from menus.

Example: ATM screens, older mobile phones, restaurant ordering kiosks.

Advantages:

- Simple to use no need to remember commands.
- Step-by-step guidance reduces user errors.

Disadvantages:

- Can be slow if menus are long or deeply nested.
- Limited flexibility compared to CLI or GUI.



Menu-Driven Interface

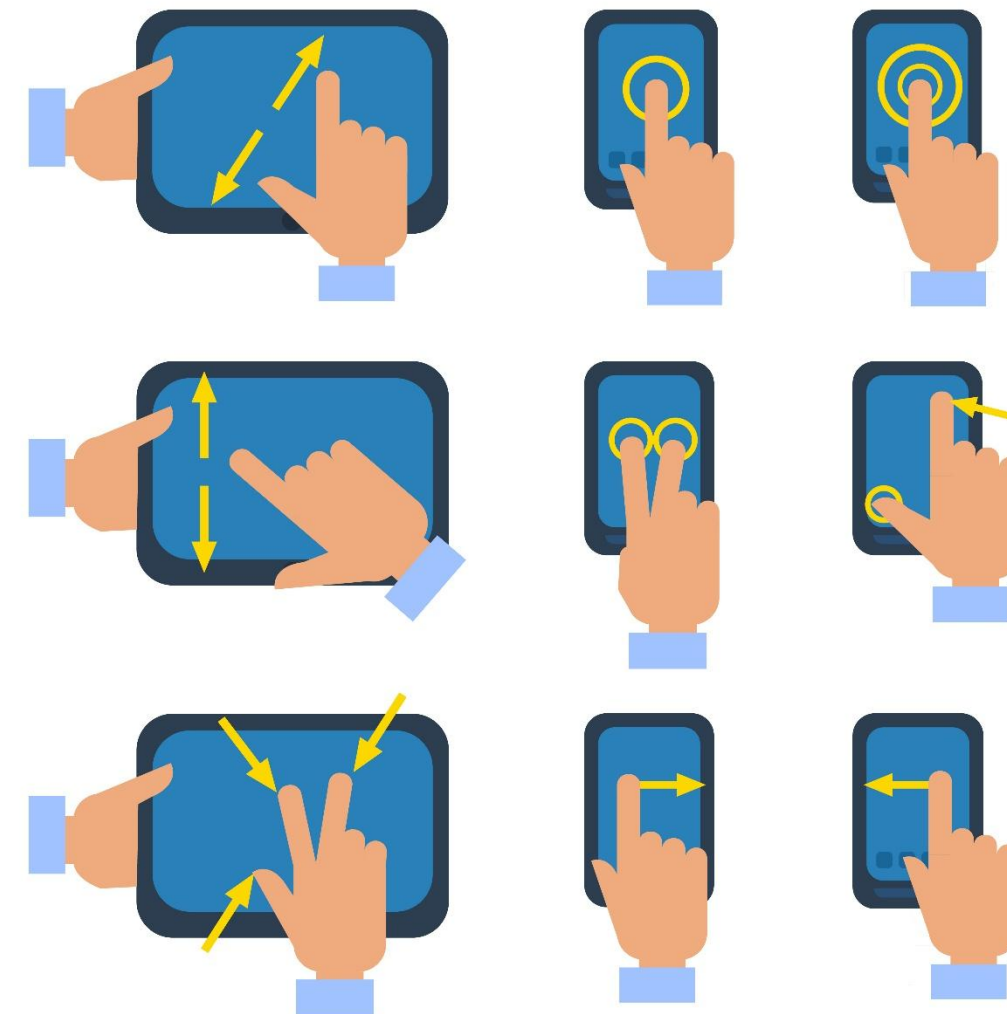
4. Touch User Interface (TUI)

Interaction happens through touch gestures like tapping, swiping, and pinching.

Example: Smartphones, tablets, interactive kiosks.

Pros: Very intuitive and fast, great for mobility.

Cons: Can get less accurate (fat fingers 🤪), not ideal for complex tasks.



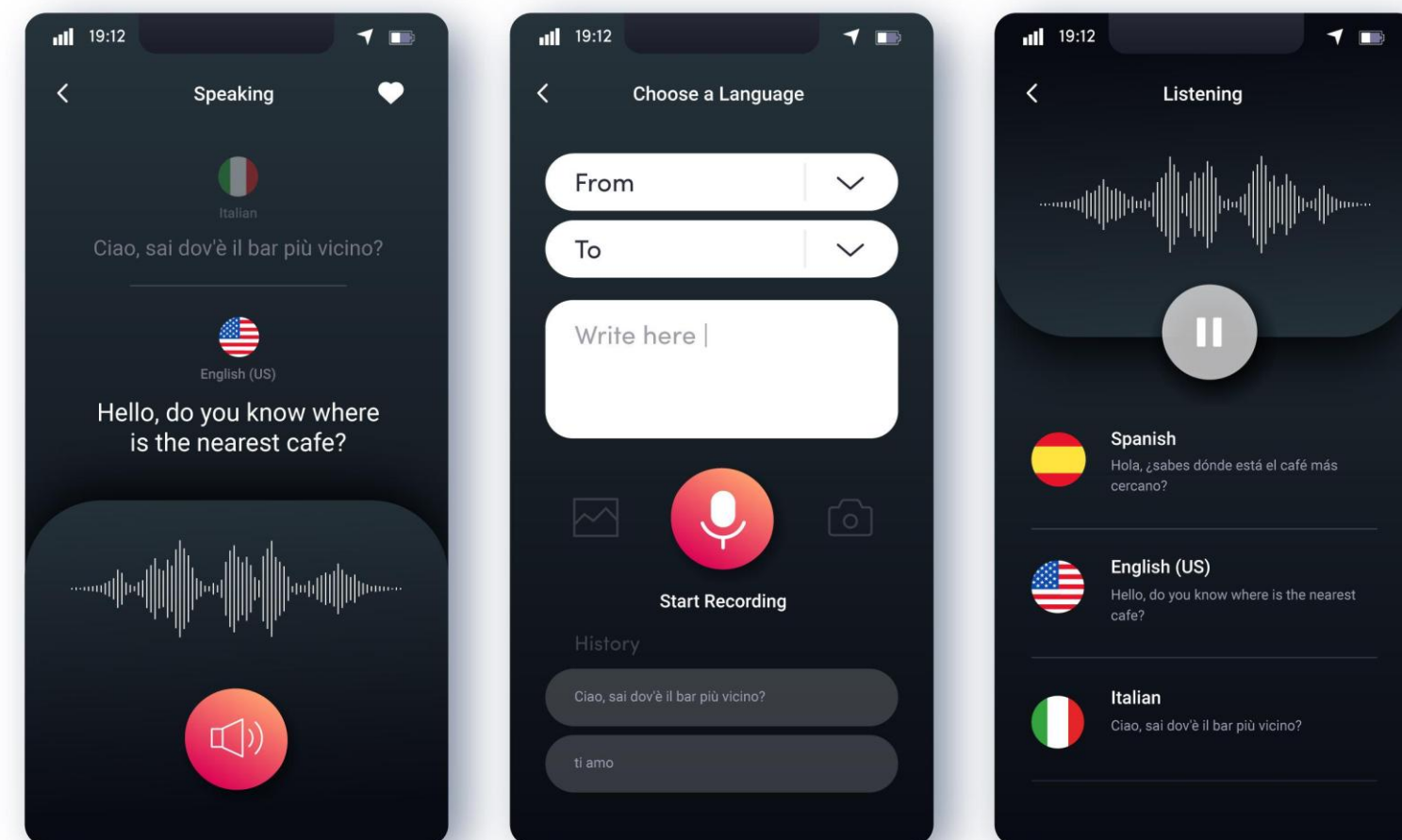
Menu-Driven Interface

5. Voice User Interface (VUI) Users interact using spoken commands.

Example: Siri, Alexa, Google Assistant.

Pros: Hands-free, accessible, convenient.

Cons: Can struggle with accents, background noise, or complex instructions.



Menu-Driven Interface

6. Natural User Interface (NUI)

Lets users interact in “natural” ways (gestures, motion, touch, voice).

Example: Xbox Kinect, VR/AR systems, gesture-controlled smart TVs. **Best for:** Gaming, immersive apps, futuristic systems.

Pros: Feels natural, immersive experience.

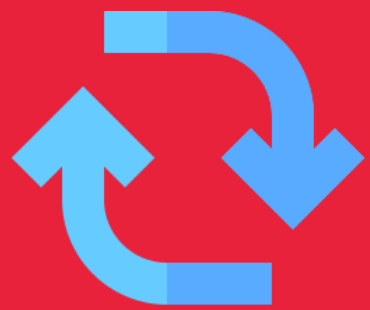
Cons: Still developing, sometimes inaccurate, needs special hardware.



OPERATING SYSTEM REQUIREMENTS

OS	Minimum Requirements	Recommended Requirements
Windows 11	- CPU: 1 GHz, 2+ cores, 64-bit- RAM: 4 GB- Storage: 64 GB- GPU: DirectX 12, WDDM 2.0- Display: 720p- TPM 2.0 + UEFI Secure Boot	- CPU: Quad-core (Intel i5/Ryzen 5 or better)- RAM: 8–16 GB- Storage: 256 GB+ SSD- GPU: Dedicated graphics (for gaming/creative work)- Display: 1080p or higher
macOS (Sonoma / Latest)	- Compatible Mac (2017+)- RAM: 8 GB- Storage: ~50 GB free- Display: Retina or 1080p	- Latest Mac with M1/M2/M3 chip- RAM: 16 GB- Storage: 256–512 GB SSD- Display: 1080p or 4K external monitor
Linux (Ubuntu)	- CPU: 2 GHz dual-core- RAM: 4 GB- Storage: 25 GB- GPU: Supports 1024x768	- CPU: Quad-core Intel/AMD- RAM: 8 GB+- Storage: 120 GB+ SSD- GPU: Dedicated graphics (optional)- Display: 1080p

Importance of Windows Update



UPDATE



Windows Update is more important than it looks at first glance it's not just about getting new features, it's really about keeping your computer safe, stable, and working smoothly.

- ◆ **Security protection** – Updates often include security patches that fix vulnerabilities hackers could exploit. Without them, your system becomes an easy target for malware, ransomware, or data breaches.
- ◆ **Bug fixes and stability** – Microsoft constantly identifies issues in Windows. Updates address crashes, glitches, and errors to make the system run more reliably.
- ◆ **Performance improvements** – Some updates are designed to optimize speed, memory usage, or battery efficiency, which helps your computer run better over time.
- ◆ **Compatibility** – Updates ensure your system can work with new hardware (printers, GPUs, etc.) and software (apps, games, drivers).
- ◆ **New features** – Microsoft occasionally adds tools or enhancements through updates, giving you extra functions without buying a new version of Windows.